Pre-training Nanochat

CSC490 Assignment 3

Janis Joplin (1009715051) • Louis Ryan Tan (1009713561)
Maverick Luke (1009714855) • Razan Ahsan Rifandi (1009562108)

January - April 2026

1 Architecture Review

1.1 Architecture Diagram

The architecture of nanochat (in initial commit) is different from GPT-2 in that it uses:

- RoPE (rotary embeddings) without learned positional embedding
- RMSNorm instead of Layer Norm with learned parameters
- QK normalization
- Untied weights for embedding and LM head
- ReLU^2 activation instead of GeLU
- Normalization after token embedding
- Multi Query Attention (MQA) instead of Multi Head Attention (MHA)

Below is the explanation of the architecture parts:

1. **Input & Token Embedding**: A tensor of integer token IDs of shape `(batch_size, sequence_length)`. These are passed through `wte`, Each integer index gets looked up and replaced with a learned vector of dimension `n_embd`. Unlike GPT-2, there are no learned positional embedding, as the positional embedding will be added later with RoPE.
2. **Transformer Block**: Each block has Multi Query Attention and MLP, both using pre-norm with residual connections.
3. **Causal self-attention**: The normalized input is projected into queries, keys, and values via three separate linear layers `(c_q, c_k, c_v)`. Q has `n_head * head_dim` output dimensions, while K and V only have `n_kv_head * head_dim`. When `n_kv_head < n_head`, multiple query heads share the same key/value heads. This reduces the KV cache size during inference. After applying RoPE and QK norm, K and V tensors are expanded so each KV head is replicated `n_head  n_kv_head` times, and the attention score is calculated.
4. **Rotary Positional Embedding**: Injects position information directly into the query and key vectors inside each attention layer using the RoPE formula.
5. **QK Norm**: After RoPE, both Q and K are individually passed through RMSNorm.
6. **MLP**: The feed-forward network expands the dimension by 4 times, applies an activation, then projects back down. The activation is ReLU^2 instead of GeLU in GPT-2.
7. **LM Head**: After all Transformer blocks, RMS Norm is applied and passed through the LM head (labeled as Linear in the diagram) and softmax is applied to get the logits.

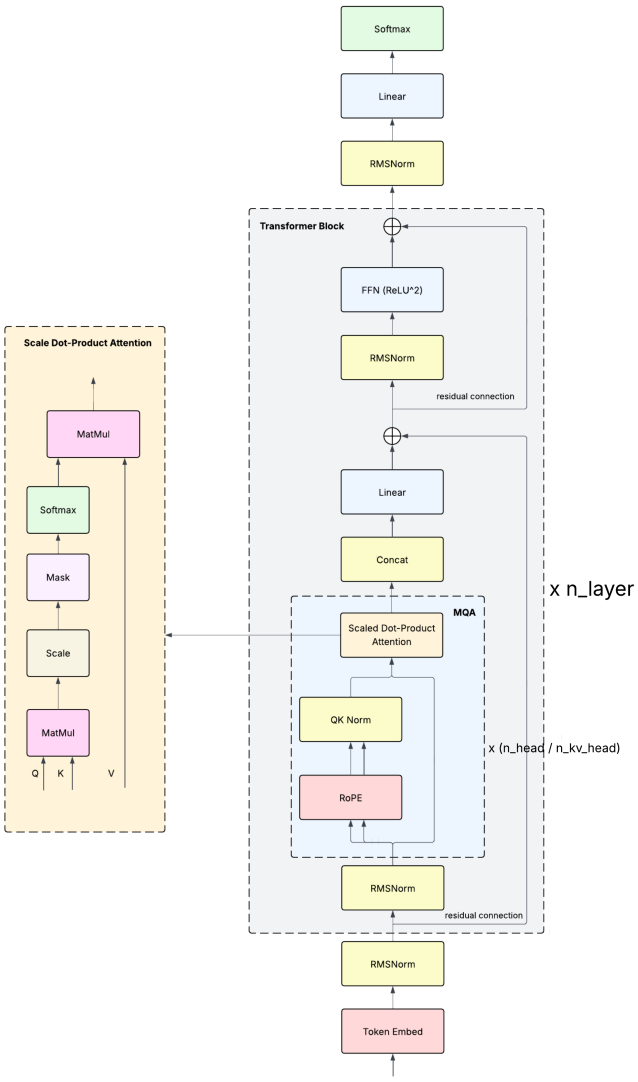

Figure 1: nanochat Architecture Diagram

1.2 Architecture Changes

Change	Motivation	Technical Details	Impact on Performance
SwiGLU Activation *Shazeer (2020) [6]; Touvron et al. (2023) [8]*	The current nanochat implementation uses ReLU^2 activations. SwiGLU is more commonly used and is the standard activation function in Llama-family (used in Llama 2 and Llama 3) and other SOTA models.	SwiGLU replaces the FFN activation (currently ReLU^2) with a gated linear unit variant: $\text{SwiGLU}(x, W, V) = \text{Swish}(xW) \otimes (xV)$, where $\text{Swish}(x) = x \cdot \sigma(x)$. The hidden dimension is scaled to $\frac{2}{3} \times 4d$ to maintain parameter parity with standard FFNs.	We expect improvement on our model based on consistent empirical improvements in perplexity and downstream task accuracy over ReLU/GELU across model sizes [6]. Smoother gradients enable more stable training.
Grouped-Query Attention (GQA) *Ainslie et al. (2023) [1]; Touvron et al. (2023) [8]*	The base parameter for nanochat has `n_head = n_kv_head`, which implies MHA is used on the attention block. Instead of MHA or MQA, major LLMs such as Llama 3 apply GQA throughout all attention layers. GQA is the preferred attention scheme for LLMs at deployment scale. Note that the code itself already supports GQA, and thus this architecture change can be done by specifying `n_kv_head < n_head` during pretraining.	GQA partitions the H query heads into G groups ($1 < G < H$), each sharing one key and value head, reducing KV-cache to G/H of MHA. MHA checkpoints are converted by mean-pooling key/value heads within each group, followed by short fine-tuning.	We expect performance on par with MHA while reducing KV-cache, yielding proportional throughput gains during decoding [1].
Mixture of Experts (MoE) FFN *Shazeer et al. (2017) [7]; Jiang et al. (2024) [2]*	Modern LLMs such as Mixtral uses Mixture of Experts, which targets dense FFN layers which dominate parameter counta nd are activated fully for every token, leaving significant compute headroom unexploited.	Each FFN block is replaced by E expert sub-networks (each a standard FFN), with a learned router selecting the top-k experts per token: $\text{MoE}(x) = \sum_{i \in \text{top-}k} g_i(x)\, \text{FFN}_i(x)$, where g_i are softmax-normalised router weights. Following Mixtral [2], E=8 and k=2 is a standard configuration, activating 2/8 experts per token and keeping active FLOPs equal to a dense model while multiplying total parameters by $\sim$4×.	Mixtral 8×7B matches or exceeds a dense 13B model on most benchmarks at the same inference cost [2]. We would expect similar performance on scale with nanochat.

2 Ablations on a nano nanochat

- Out of the 3 architecture changes in Part 1, we chose to test and implement **SwiGLU** and **GQA**.
- The base version of nanochat (referencing the GPT-2-grade model trained in `speedrun.sh`) used a depth of 24, while the model trained from `quick_test` in the course tutorial's `nanochat_modal.py` used a depth of 12 (GPT-1-grade). The CLI commands we used for training the models are:

```
modal run nanochat_modal::stage_pretrain.py \
      --depth=16 \
      --wandb-run=picochat-base-d16
modal run nanochat_modal::stage_pretrain.py \
      --depth=16 \
      --use-swiglu \
      --wandb-run=picochat-d16-swiglu
modal run nanochat_modal::stage_pretrain.py \
      --depth=16 \
      --n-kv-head-ratio=2 \
      --wandb-run=picochat-d16-
```

  Taking the balance of training time and model quality into consideration, we decided that our picochat will use a depth of 16 (16 layers, 536,872,992 parameter count).

- We trained 3 models, picochat base, picochat with SwiGLU, and picochat with GQA (2:1 Q to KV head ratio). We specified a batch size of 16 and trained 1 epoch over 160 shards of the dataset. All models we trained using 8xH100 GPU and took approximately 20 minutes to pretrain which nets, and thus each model costs around $10.53 to train on Modal. The table summarizing the results can be found below, while the training visualization can be found in this W&B report: https://api.wandb.ai/links/contribnow-none/w212avn0.

Table 2: Picochat Ablation Results

Model	**Val BPB**	**Train Loss**	**Train Time (s)**	**Core Metric**
picochat-base-d16	0.79313	2.59374	1150	0.1995
picochat-d16-swiglu	0.79679	2.6053	1223	0.186
picochat-d16-gqa2	0.80056	2.6633	1005	0.186

- From the table, we can see that SwiGLU takes longer to train while GQA is faster to train when compared to baseline. Loss, validation BPB, and core metric values are similar for all models, with GQA being the worst performing model with negligible difference ($+0.007$ validation BPB, $+0.07$ train loss, -0.135 core metric compared to baseline). From the results, we conclude that GQA provides faster training time when compared to baseline with no significant loss in quality. We can see that GQA training is around 12.5% faster when compared to baseline, which will save compute costs and time when scaled to the final model training. On the other hand, the SwiGLU model had very similar performance to baseline and takes around 6.5% longer to train. We conclude that out of the 2 changes, GQA could be considered for the final model while SwiGLU should not as it provides no meaningful difference. As we stopped considering SwiGLU, we did not perform ablation testing on a model implementing both SwiGLU and GQA to save compute resources
- For a larger run training nanochat on depth 24, we would expect similar ratio of speedup utilizing GQA-2. A training time of 3 hours using 8xH100 GPUs on the base GPU will save $12.5\% * 180$ minutes $= 22.5$ minutes which saves $11.85 on training the full model.

3 Extending the context window

3.1 Training and Evaluation Setup

First, we trained our picochat model (`depth = 16`) for 1,200 steps with a shorter context window ($T = 512$). After this stage finished, we saved a checkpoint (ckpt1). We then resumed training from this checkpoint and continued training for another 1,200 steps with a longer context window ($T = 2,048$), producing a second checkpoint (ckpt2). The final training therefore consisted of a total of 2,400 optimization steps, approximately 51% of the number of steps run by the original nanochat, which is 4,704 steps. To further reduce computational cost, we trained on only a portion of the dataset using `num_shards = 160`.

We justify this choice of sequence length and training dataset size based on [5] (GPT-1), which used a sequence length of 512 and demonstrated that this sequence length is sufficient for the model to learn core syntactic and semantic structure. This fits picochat as with 16 layers, its model size lies between GPT-1 and GPT-2, in which the latter is trained using a sequence length of 1,024. Furthermore, we justify short-context pretraining followed by training on an extended context window based on the results of [4]. Their framework shows that models trained on shorter sequences can generalize to longer sequences at inference.

We evaluated both checkpoints using the `base_eval` pipeline, which reports results on the CORE and BPB benchmarks. To compare the two checkpoints, we evaluated ckpt1 (step 1,200) and ckpt2 (step 2,400) using the same base evaluation configuration. This allows us to measure the impact of continuing training with the longer context window. Our main evaluation metric is the CORE centered score, which aggregates the centered score from 22 different tasks. Although accuracy is also reported, we focus primarily on the centered score. Accuracy depends on the number of answer options provided in each task and is therefore not always directly comparable across different tasks. In contrast, the centered score measures performance relative to random guessing, making it a more consistent metric for comparing model performance across tasks.

For reproducibility, we provide the commands used to run the training and evaluation stages. The first command launches the two-stage training process (short-context followed by extended-context training), and the second command evaluates both checkpoints using the base evaluation pipeline.

```
modal run nanochat_part3.py::part3 \
  --depth 16 \
  --num-shards 160 \
  --short-seq-len 512 \
  --long-seq-len 2048 \
  --short-num-iterations 1200 \
  --long-additional-iterations 1200 \
  --short-wandb-run "picochat-part3-d16-base-shortctx" \
  --long-wandb-run "picochat-part3-d16-base-extendctx"
```

```
modal run nanochat_part3.py::stage_part3_eval_compare \
  --model-tag d16-relu2-ctxext \
  --checkpoint1-step 1200 \
  --checkpoint2-step 2400
```

3.2 Results and Discussion

We compare three models: the short-context checkpoint (ckpt1, step 1,200, $T = 512$), the extended-context checkpoint (ckpt2, step 2,400, $T = 2,048$), and the baseline picochat model trained for 4,704 steps. Table 3 reports selected centered CORE scores.

Task	ckpt1 (1200)	ckpt2 (2400)	baseline (4704)
`hellaswag_zeroshot`	0.151111	0.213333	0.233752
`arc_challenge`	0.084444	0.151111	0.086462
`piqa`	0.353333	0.420000	0.391730
CORE centered	0.143873	0.165574	0.190356

Table 3: Comparison of selected centered scores across checkpoints and the baseline picochat model.

Continuing training with the longer context window improves performance over the short-context checkpoint. The aggregate CORE centered score increases from 0.143873 to 0.165574, a gain of 0.021701. Several individual

tasks also improve, including `hellaswag_zeroshot` (0.151111 → 0.213333), `arc_challenge` (0.084444 → 0.151111), and `piqa` (0.353333 → 0.420000). This suggests that resuming training with a larger context window improves overall capability relative to stopping after the short-context stage.

We also compare this model (ckpt2) to the baseline picochat model trained for 4,704 steps. The baseline remains stronger overall on the aggregate CORE metric (0.190356 versus 0.165574), which is expected given its larger training budget. However, the ckpt2 model is competitive on some tasks and even exceeds the baseline on `arc_challenge` and `piqa`. This suggests that the shorter-then-longer context curriculum can still produce reasonable task performance despite using fewer total training steps.

From the Weights & Biases logs, the first 1,200 short-context steps required 351.3456 seconds, and the continued 1,200 long-context steps required 527.70447 seconds, for a total of 879.05007 seconds. In contrast, the baseline picochat model required 1,150.23347 seconds. This means the curriculum run used less total training time, which would also translate to lower GPU cost on Modal.

The training loss also supports this trade-off. After 1,200 short-context steps, the training loss was 2.9747, and after continuing with the 2,048-token context it decreased to 2.71204. The baseline picochat model had training loss 2.81392 at the same stage and reached 2.59374 only after the full 4,704 steps. Similarly, validation BPB improved from 0.90595 after the short-context stage to 0.82988 after the long-context continuation, while the baseline achieved 0.79313 after full training. Therefore, although the baseline performs better overall, the proposed approach achieves reasonably comparable loss, BPB, and benchmark performance with relatively lower total training time.

For completeness and reproducibility, the full training logs and metrics are available in our Weights & Biases report: https://api.wandb.ai/links/contribnow-none/kbwnadzb. This report includes detailed training curves, loss values, validation BPB, and runtime statistics for all experiments described in this section.

3.3 Additional Training Experiments

We conducted an additional experiment with different parameters. Rather than splitting training evenly between the two context lengths (1,200 short + 1,200 extended steps), we extended the short-context phase to 3,500 steps, followed by the remaining steps of 1 epoch (around 4,700 total steps) using the extended context window. We also increased the batch size from 16 to 64 during the short-context phase. We predicted that the reduced sequence length and higher batch size would allow faster per-step processing and thus yield a speedup. However, the training time for the short-context phase was on par with the original picochat baseline.

From these results, we initially hypothesized that the workload was CPU-bound. To test this, we reduced the GPU allocation from 8xH100 to 4xH100. Under this, training time roughly doubled while short-context training time still matches that of the baseline.

We therefore attribute the slow training to hardware-level inefficiencies when the attention kernels and memory access patterns are not optimized for the short sequence length. Modern GPU architectures and libraries such as FlashAttention are tuned for larger sequence lengths, and operating at $T = 512$ with a large batch size may underutilize available memory bandwidth or introduce padding overhead that negates the theoretical compute savings. In contrast, $T = 2,048$ is more widely used and thus is more optimized.

These suggest that the wall-clock efficiency gains from short-context pretraining are not necessarily present in practice. The visualization for these additional training experiments can be found in this W&B report: https://api.wandb.ai/links/contribnow-none/etoo4oln

4 Training Your Final Nanochat

4.1 Nanochat Configuration

The final nanochat model uses the following training configuration

- `depth`: 24
- `num_shards`: 240
- `n_kv_head_ratio`: 2

The total parameter count is $1,025,509,680$. All remaining hyperparameters are kept at their default values.

We justify the given set of hyperparameter based on the original parameters used in the `speedrun.sh` script in the original nanochat repository. The model will have We chose to implement GQA with KV head ratio 2 based on the results on our ablation studies in Part 2, expecting shorter training time with similar performance compared to MHA. We chose not to implement SwiGLU as it does not give significant performance gain when compared to baseline (ReLU^2). For Part 3, we chose not to implement short-context pretraining even though the evaluation is higher than baseline for some task since, in general, the CORE centered score of the model trained using the method in Part 3 is worse than baseline.

4.2 Scaling Prediction

The nanochat model was trained with 1.5× more depth and 1.5× more data shards. The get an estimate from improvement, we adopt the Kaplan-style scaling law and model the relative improvement as follows

$$\frac{L_{\text{nanochat}}}{L_{\text{picochat}}} \propto \left(\frac{N_{\text{nanochat}}}{N_{\text{picochat}}}\right)^{-\alpha} \propto \left(\frac{d_{\text{nanochat}}}{d_{\text{picochat}}}\right)^{-3\alpha} = 1.5^{-3\alpha}$$

Using the fitted alpha value from Kaplan (2020) [3], $\alpha \approx 0.076$, we predict a $\approx 8.85\%$ decrease in training loss.

The actual training losses that we observe are $L_{\text{picochat}} = 2.6630$ and $L_{\text{nanochat}} = 2.3575$, which corresponds to a relative drop in training loss by 11.4%. This empirical improvement is slightly larger than the scaling-law prediction but remains within the same order of magnitude, suggesting that the Kaplan scaling relationship provides a reasonable approximation even at the smaller model scale considered here.

More sophisticated scaling laws, such as the Chinchilla formulation, explicitly account for both model size and training data. However, because the models in this assignment are trained at a much smaller scale than those used to fit the Chinchilla law, we opted not to apply those coefficients directly and instead used the simpler Kaplan-style approximation.

4.3 Results and Summary

The pretraining run visualization can be found in this W&B report: https://api.wandb.ai/links/contribnow-none/ku9cyfcl. Below is the table summarizing the results of the fine-tuned final model.

Table 4: Final Model Benchmark Results

Benchmark	**Base Model (pre-SFT)**	**SFT Step 200**	**SFT Step 400**	**SFT Final (Step 485)**
ARC-Easy	69.57%	57.66%	62.25%	63.55%
ARC-Challenge	38.05%	43.94%	48.89%	49.49%
MMLU	—	36.42%	37.39%	37.47%
GSM8K	—	4.17%	16.67%	9.02%
HumanEval	—	0.00%	8.33%	8.33%
SpellingBee	—	100.00%	100.00%	100.00%
Val BPB	—	0.3074	0.2835	0.2774
ChatCORE	—	0.3137	0.3717	0.3551
ChatCORE (cat)	—	0.2801	0.3268	0.3352

4.4 Emergent Behavior

The list of 10 questions we cane up with are as follows:

```
1. If you put a metal spoon in a hot bowl of soup, why does the handle get warm?
2. A plant is kept in a dark closet for two weeks. What will happen to it and why?
3. What is the difference between a hypothesis and a theory in science?
4. If the Earth had no moon, how would ocean tides be different?
5. You have 3 bags with 4 apples each. You eat 5. How many are left?
6. Which of these is a conductor of electricity: rubber, copper, wood, or plastic?
7. Why do we see lightning before we hear thunder?
8. I'm feeling cold. Should I put on a sweater or open a window?
9. Continue this pattern: 2, 4, 8, 16, ___
10. Alice had 10 apples and Bob had 3 apples. How many apples do Bob and Alice both have?
```

From the results we got from picochat and nanochat, nanochat seemed to gained the ability to do factual reasoning and basic scientific cause-and-effect. picochat's output for the questions are largely incoherent and the model seem to simply learn language patterns than actual reasoning, while nanochat's output seem to have basic causal reasoning in it. For example, for Question 1, picochat's output is `i dont́ know.` while nanochat's output is `The heat inside the bowl causes the food to get hot`. This is also reflected by nanochat's decent performance on the ARC (A12 Reasoning Challenge) Benchmark. nanochat attains 63.55% and 49.49% in ARC Easy and Challenge respectively, while picochat only attains 46.93% and 37.03%.

References

[1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. *arXiv:2305.13245*, 2023.

[2] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Dahan, et al. Mixtral of Experts. *arXiv:2401.04088*, 2024.

[3] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

[4] Ofir Press, Noah A. Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. In *International Conference on Learning Representations*, 2022.

[5] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding by generative pre-training. 2018.

[6] Noam Shazeer. GLU Variants Improve Transformers. *arXiv:2002.05202*, 2020.

[7] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. *arXiv:1701.06538*, 2017.

[8] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open Foundation and Fine-Tuned Chat Models. *arXiv:2307.09288*, 2023.